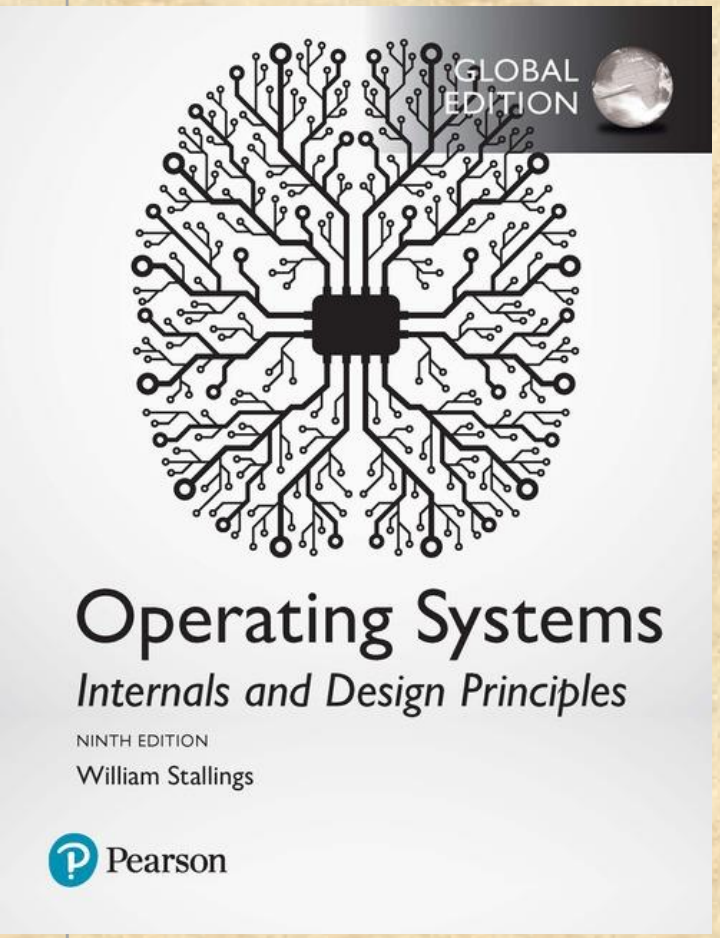


Chapter 10 Multiprocessor, Multicore and Real-Time Scheduling

Ninth Edition, Global Edition
By William Stallings

Real-Time Systems

- The operating system, and in particular the scheduler, is perhaps the most important component

Examples:

- Control of laboratory experiments
- Process control in industrial plants
- Robotics
- Air traffic control
- Telecommunications
- Military command and control systems

- Correctness of the system depends not only on the logical result of the computation but also on the time at which the results are produced
- Tasks or processes attempt to control or react to events that take place in the outside world
- These events occur in "real time" and tasks must be able to keep up with them

Hard and Soft Real-Time Tasks

Hard real-time task

- One that must meet its deadline
- Otherwise it will cause unacceptable damage or a fatal error to the system

Soft real-time task

- Has an associated deadline that is desirable but not mandatory
- It still makes sense to schedule and complete the task even if it has passed its deadline

Periodic and Aperiodic Tasks

- **Periodic tasks**
  - Requirement may be stated as:
    - Once per period T
    - Exactly T units apart
- **Aperiodic tasks**
  - Has a deadline by which it must finish or start
  - May have a constraint on both start and finish time

Characteristics of Real Time Systems

Real-time operating systems have requirements in five general areas:

- Determinism
- Responsiveness
- User control
- Reliability
- Fail-soft operation

Determinism

- Concerned with how long an operating system delays before acknowledging an interrupt
- Operations are performed at fixed, predetermined times or within predetermined time intervals
  - When multiple processes are competing for resources and processor time, no system will be fully deterministic

The extent to which an operating system can deterministically satisfy requests depends on:

- The speed with which it can respond to interrupts
- Whether the system has sufficient capacity to handle all requests within the required time

Responsiveness

- Together with determinism make up the response time to external events
  - Critical for real-time systems that must meet timing requirements imposed by individuals, devices, and data flows external to the system
- Concerned with how long, after acknowledgment, it takes an operating system to service the interrupt

Responsiveness includes:

- Amount of time required to initially handle the interrupt and begin execution of the interrupt service routine (ISR)
- Amount of time required to perform the ISR
- Effect of interrupt nesting

User Control

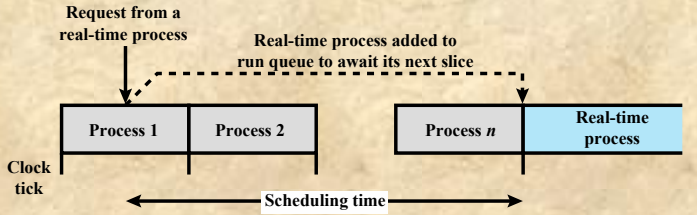
- Generally much broader in a real-time operating system than in ordinary operating systems
- It is essential to allow the user fine-grained control over task priority
- User should be able to distinguish between hard and soft tasks and to specify relative priorities within each class
- May allow user to specify such characteristics as:

Reliability

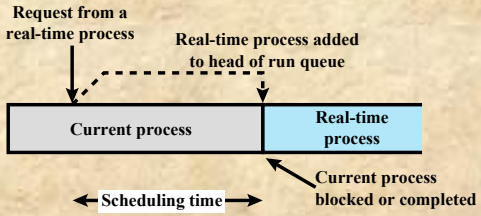
- More important for real-time systems than non-real time systems
- Real-time systems respond to and control events in real time so loss or degradation of performance may have catastrophic consequences such as:
  - Financial loss
  - Major equipment damage
  - Loss of life

Fail-Soft Operation

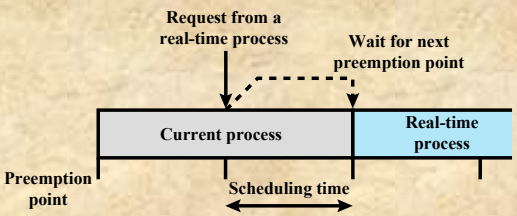
- A characteristic that refers to the ability of a system to fail in such a way as to preserve as much capability and data as possible
- Important aspect is stability
  - A real-time system is stable if the system will meet the deadlines of its most critical, highest-priority tasks even if some less critical task deadlines are not always met
- The following features are common to most real-time OSs
  - A stricter use of priorities than in an ordinary OS, with preemptive scheduling that is designed to meet real-time requirements
  - Interrupt latency is bounded and relatively short
  - More precise and predictable timing characteristics than general purpose OSs



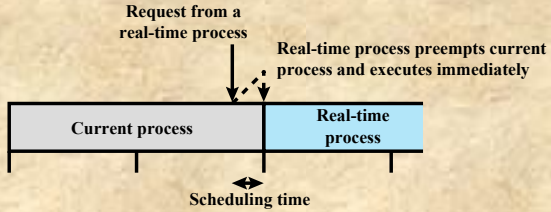
(a) Round-robin Preemptive Scheduler



(b) Priority-Driven Nonpreemptive Scheduler



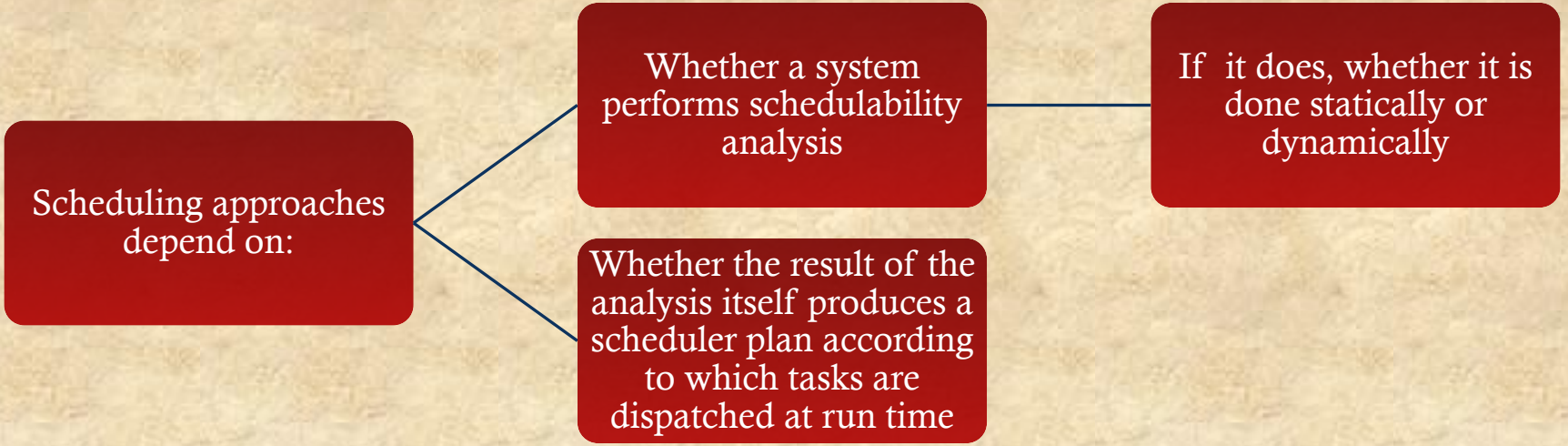
(c) Priority-Driven Preemptive Scheduler on Preemption Points



(d) Immediate Preemptive Scheduler

Figure 10.4 Scheduling of Real-Time Process

Real-Time Scheduling



Classes of Real-Time Scheduling Algorithms

Static table-driven approaches

- Performs a static analysis of feasible schedules of dispatching
- Result is a schedule that determines, at run time, when a task must begin execution

Static priority-driven preemptive approaches

- A static analysis is performed but no schedule is drawn up
- Analysis is used to assign priorities to tasks so that a traditional priority-driven preemptive scheduler can be used

Dynamic planning-based approaches

- Feasibility is determined at run time rather than offline prior to the start of execution
- One result of the analysis is a schedule or plan that is used to decide when to dispatch this task

Dynamic best effort approaches

- No feasibility analysis is performed
- System tries to meet all deadlines and aborts any started process whose deadline is missed

Scheduling Algorithms

Static table-driven scheduling

- Applicable to tasks that are periodic
- Input to the analysis consists of the periodic arrival time, execution time, periodic ending deadline, and relative priority of each task
- This is a predictable approach but one that is inflexible because any change to any task requirements requires that the schedule be redone
- Earliest-deadline-first or other periodic deadline techniques are typical of this category of scheduling algorithms

Static priority-driven preemptive scheduling

- Makes use of the priority-driven preemptive scheduling mechanism common to most non-real-time multiprogramming systems
- In a non-real-time system, a variety of factors might be used to determine priority
- In a real-time system, priority assignment is related to the time constraints associated with each task
- One example of this approach is the rate monotonic algorithm which assigns static priorities to tasks based on the length of their periods

Scheduling Algorithms

Dynamic planning-based scheduling

- After a task arrives, but before its execution begins, an attempt is made to create a schedule that contains the previously scheduled tasks as well as the new arrival
- If the new arrival can be scheduled in such a way that its deadlines are satisfied and that no currently scheduled task misses a deadline, then the schedule is revised to accommodate the new task

Dynamic best effort scheduling

- The approach used by many real-time systems that are currently commercially available
- When a task arrives, the system assigns a priority based on the characteristics of the task
- Some form of deadline scheduling is typically used
- Typically the tasks are aperiodic so no static scheduling analysis is possible
- The major disadvantage of this form of scheduling is, that until a deadline arrives or until the task completes, we do not know whether a timing constraint will be met
- Its advantage is that it is easy to implement

Deadline Scheduling

- Real-time operating systems are designed with the objective of starting real-time tasks as rapidly as possible and emphasize rapid interrupt handling and task dispatching
- Real-time applications are generally not concerned with sheer speed but rather with completing (or starting) tasks at the most valuable times
- Priorities provide a crude tool and do not capture the requirement of completion (or initiation) at the most valuable time

Information Used for Deadline Scheduling

Ready time	• Time task becomes ready for execution
Starting deadline	• Time task must begin
Completion deadline	• Time task must be completed
Processing time	• Time required to execute the task to completion
Resource requirements	• Resources required by the task while it is executing
Priority	• Measures relative importance of the task
Subtask scheduler	• A task may be decomposed into a mandatory subtask and an optional subtask

Table 10.3
Execution Profile of Two Periodic Tasks

Process	Arrival Time	Execution Time	Ending Deadline
A(1)	0	10	20
A(2)	20	10	40
A(3)	40	10	60
A(4)	60	10	80
A(5)	80	10	100
• • •	• • •	• • •	• • •
B(1)	0	25	50
B(2)	50	25	100
• • •	• • •	• • •	• • •

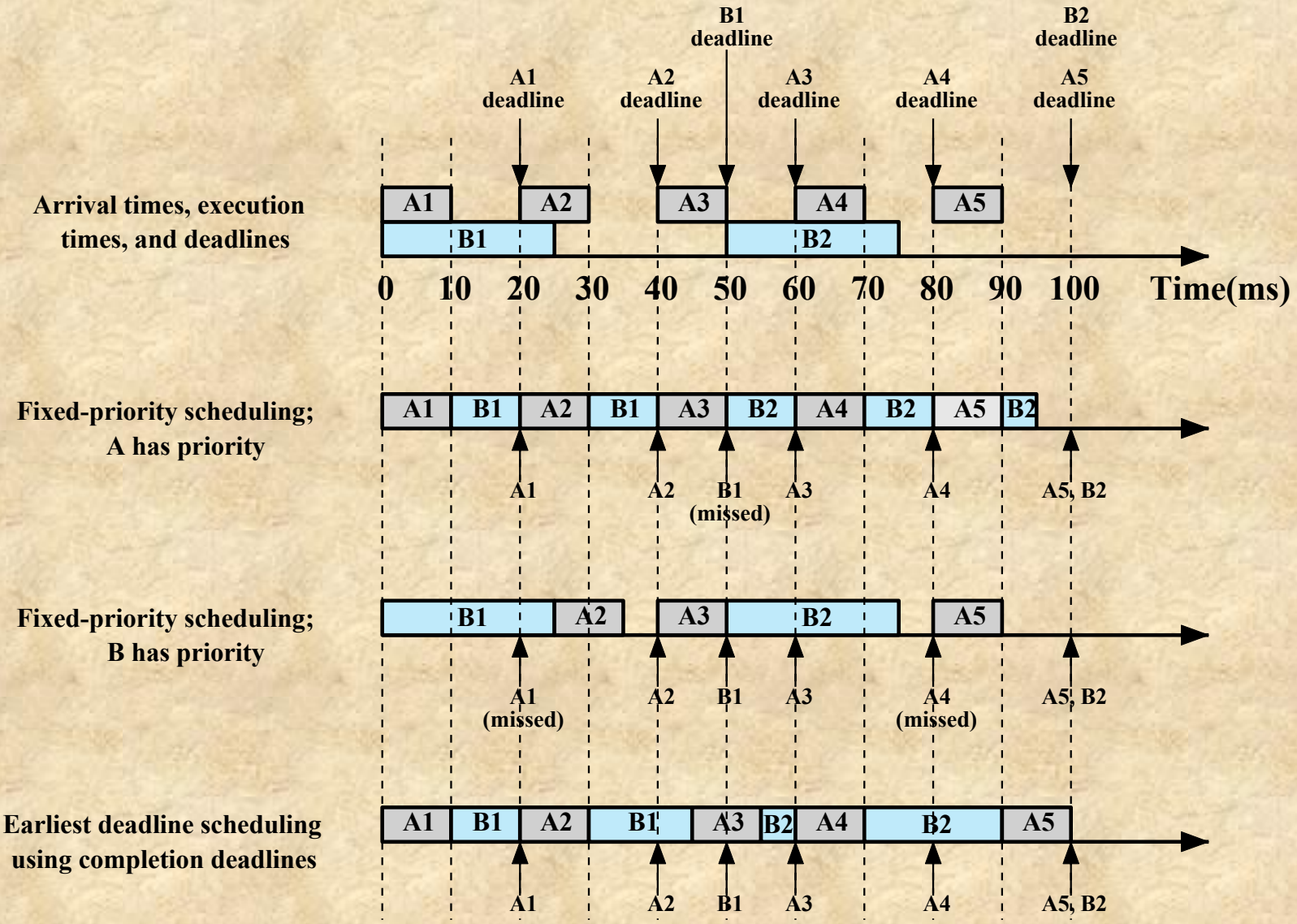


Figure 10.5 Scheduling of Periodic Real-time Tasks with Completion Deadlines (based on Table 10.3)

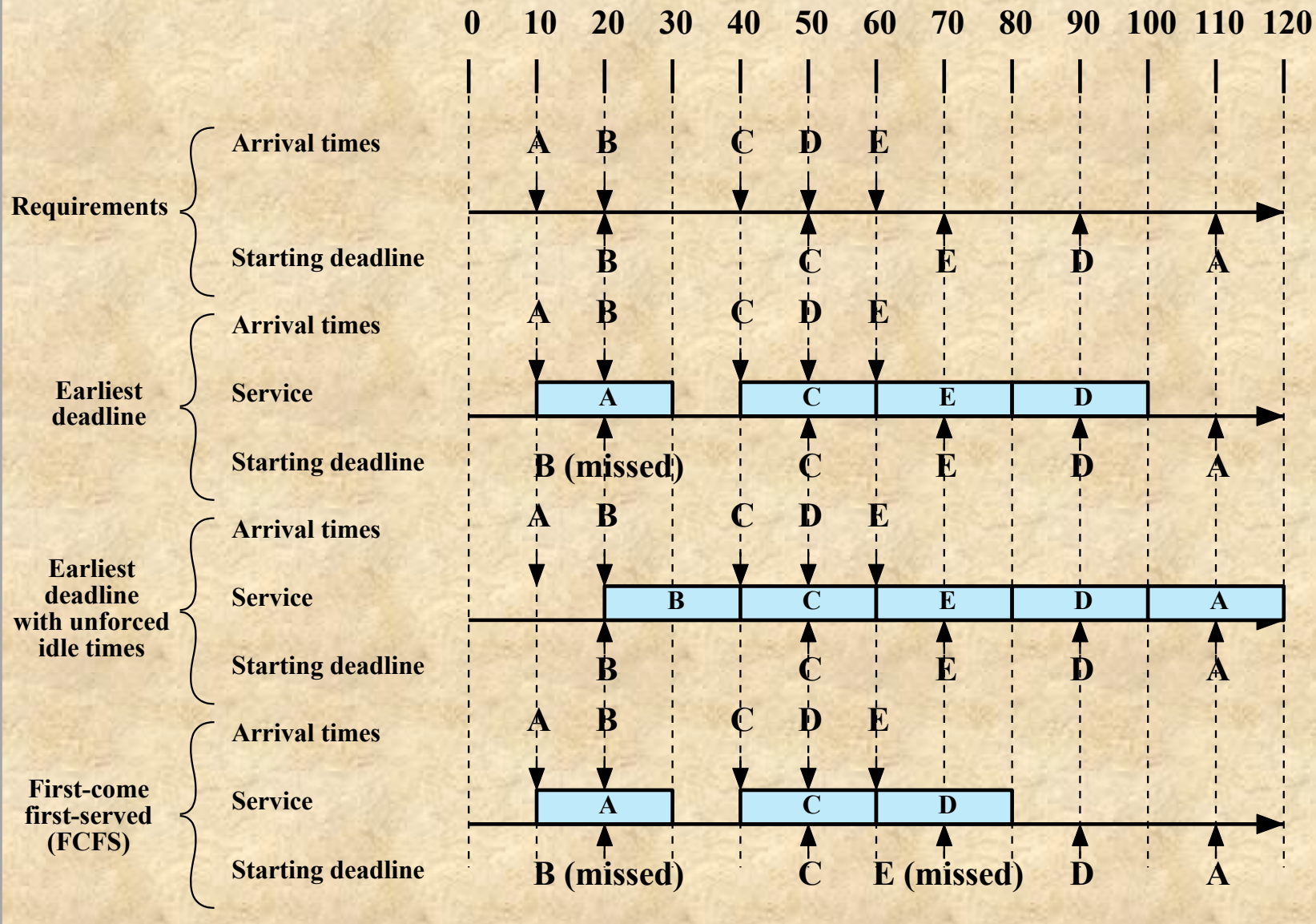


Figure 10.6 Scheduling of Aperiodic Real-time Tasks with Starting Deadlines

Table 10.4
Execution Profile of Five Aperiodic Tasks

Process	Arrival Time	Execution Time	Starting Deadline
A	10	20	110
B	20	20	20
C	40	20	50
D	50	20	90
E	60	20	70

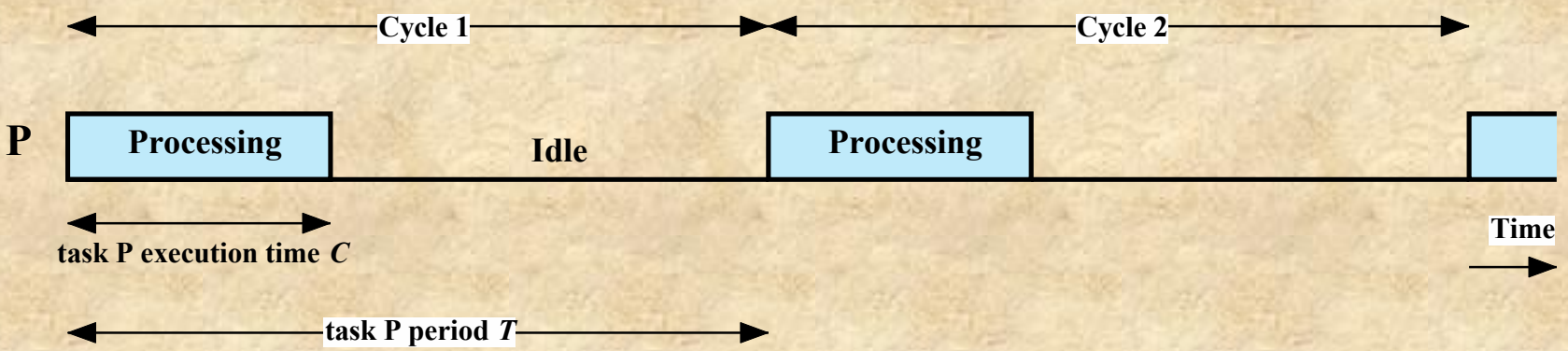
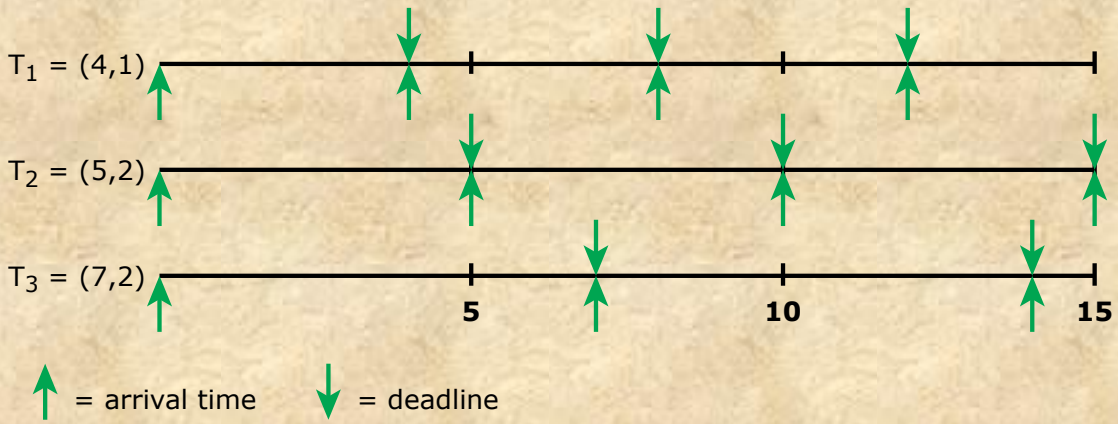
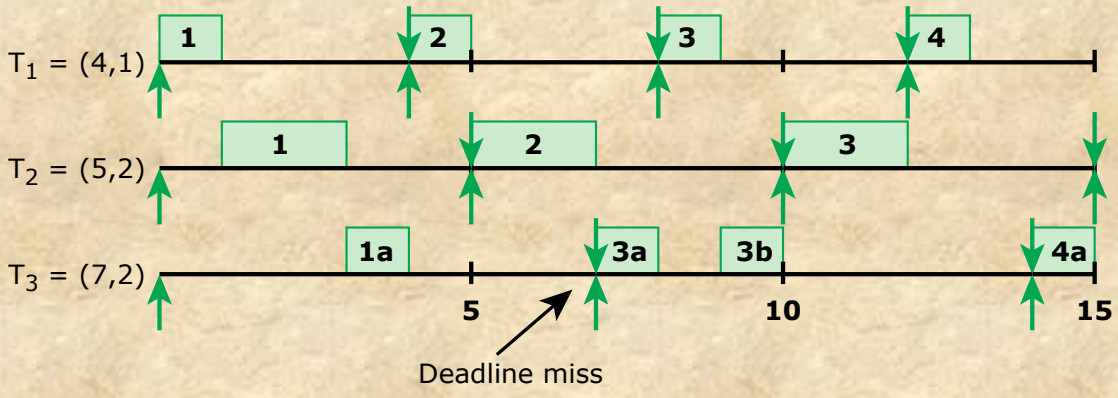


Figure 10.7 Periodic Task Timing Diagram



(a) Arrival times and deadines for task $T_i = (P_i, C_i)$; P_i = period, C_i = processing time



(b) Scheduling results

Figure 10.8 Rate Monotonic Scheduling Example

Table 10.5

Value of the RMS Upper Bound

n	$n(2^{1/n} - 1)$
1	1.0
2	0.828
3	0.779
4	0.756
5	0.743
6	0.734
•	•
•	•
•	•
□	ln 2 □ 0.693

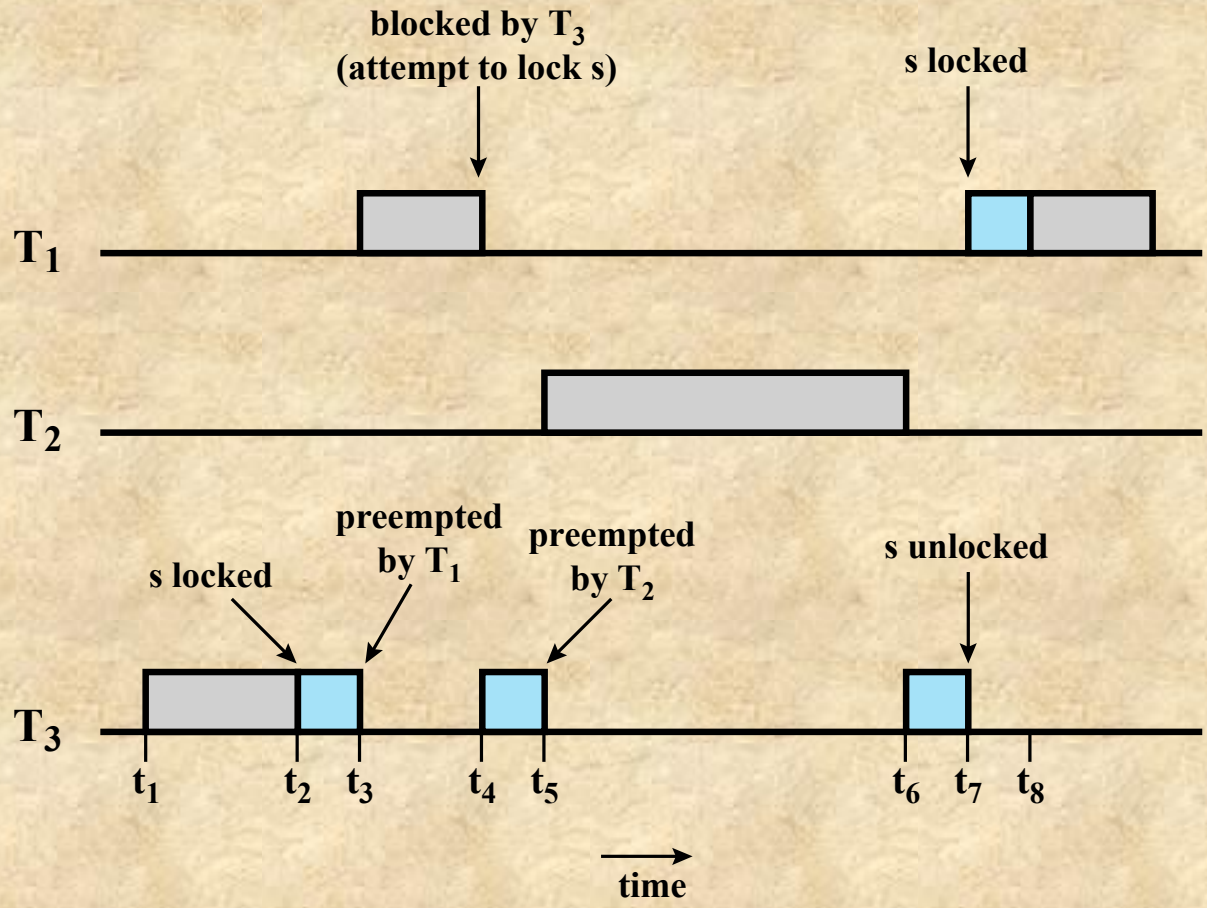
Priority Inversion

- Can occur in any priority-based preemptive scheduling scheme
- Particularly relevant in the context of real-time scheduling
- Best-known instance involved the Mars Pathfinder mission
- Occurs when circumstances within the system force a higher priority task to wait for a lower priority task

Unbounded Priority Inversion

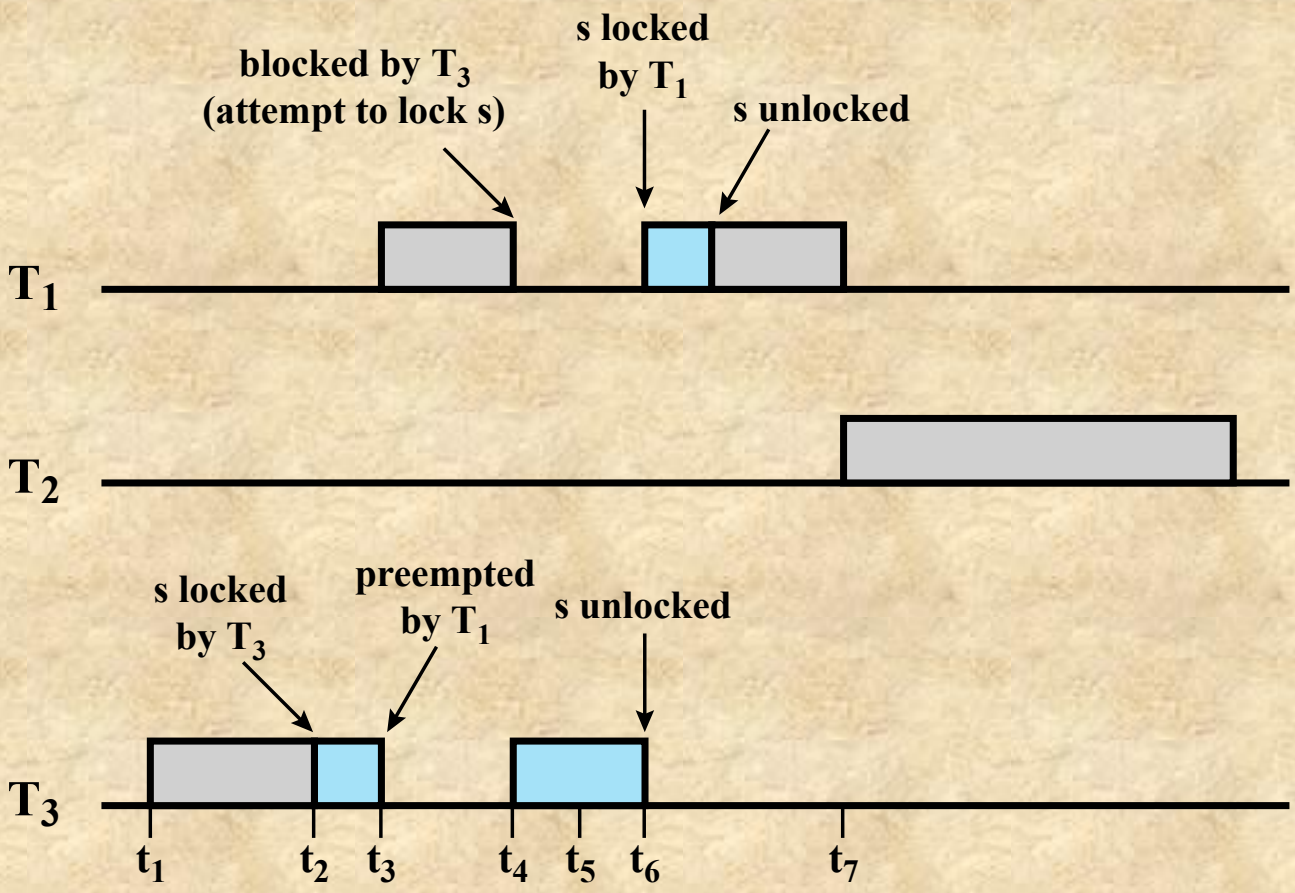
- The duration of a priority inversion depends not only on the time required to handle a shared resource, but also on the unpredictable actions of other unrelated tasks

Unbounded Priority Inversion



(a) Unbounded priority inversion

Priority Inheritance



(b) Use of priority inheritance

Linux Scheduling

- The three primary Linux scheduling classes are:
  - SCHED_FIFO: First-in-first-out real-time threads
  - SCHED_RR: Round-robin real-time threads
  - SCHED_NORMAL: Other, non-real-time threads
- Within each class multiple priorities may be used, with priorities in the real-time classes higher than the priorities for the SCHED_NORMAL class

A	minimum
B	middle
C	middle
D	maximum

(a) Relative thread priorities

(b) Flow with FIFO scheduling

(c) Flow with RR scheduling

Figure 10.10 Example of Linux Real-Time Scheduling

Summary

- Multiprocessor and multicore scheduling
  - Granularity
  - Design issues
  - Process scheduling
  - Thread scheduling
  - Multicore thread scheduling
- Linux scheduling
  - Real-time scheduling
  - Non-real-time scheduling
- UNIX FreeBSD scheduling
  - Priority classes
  - SMP and multicore support
- Real-time scheduling
  - Background
  - Characteristics of real-time operating systems
  - Real-time scheduling
  - Deadline scheduling
  - Rate monotonic scheduling
  - Priority inversion
- Windows scheduling
  - Process and thread priorities
  - Multiprocessor scheduling
- UNIX SVR4 scheduling